



Consigli per la gestione UX nel web dev

Python

Quando si realizza una pagina web non si sta semplicemente “scrivendo codice”, ma si sta progettando un sistema di comunicazione visiva e interattiva.

Ogni elemento presente in una pagina, testi, colori, pulsanti, immagini, spaziature, menu e animazioni, contribuisce all’esperienza finale dell’utente.

Una buona pagina web deve quindi rispettare due aspetti fondamentali:

- essere chiara e piacevole da vedere;
- essere semplice e intuitiva da utilizzare.



Nel web development moderno questi due aspetti vengono identificati principalmente con i concetti di UI e UX, due discipline strettamente collegate ma con obiettivi differenti.

Cos'è la UI (User Interface)

La UI (User Interface) rappresenta l'interfaccia grafica con cui l'utente interagisce; è tutta la parte visiva e concreta della pagina web.

La UI comprende elementi come:

- layout della pagina;
- colori;
- tipografia;
- pulsanti;
- menu di navigazione;
- icone;
- immagini;
- form;
- spaziature;
- animazioni e transizioni.



Cos'è la UX (User Experience)

La UX (User Experience) rappresenta invece l'esperienza complessiva dell'utente durante l'utilizzo del sito o dell'applicazione.

La UX non riguarda solamente l'aspetto grafico, ma soprattutto:

- facilità d'uso;
- logica della navigazione;
- velocità nel trovare informazioni;
- accessibilità;
- semplicità delle operazioni;
- chiarezza dei percorsi;
- riduzione degli errori;
- comfort generale durante l'utilizzo.



L'obiettivo della UX è fare in modo che l'utente riesca a svolgere le proprie azioni nel modo più naturale e intuitivo possibile.

In pratica, la UX si occupa di come funziona e come viene vissuto il sito.

Ad esempio:

- un checkout semplice in un e-commerce;
- un menu intuitivo;
- pochi passaggi per registrarsi;
- una struttura chiara delle informazioni;

sono aspetti legati alla User Experience.



Differenza tra UI e UX

UI e UX lavorano insieme ma non sono la stessa cosa.

- La UI riguarda l'aspetto grafico e visivo.
- La UX riguarda l'esperienza e la facilità d'utilizzo.

Un sito può:

- essere molto bello graficamente (buona UI),
- ma difficile da usare (cattiva UX).

Oppure:

- essere semplice e funzionale (buona UX),
- ma poco curato visivamente (UI debole).



Un buon progetto web cerca sempre di unire entrambe le componenti.

Feedback immediato all'utente

Un buon principio di UX è: quando l'utente fa qualcosa, l'applicazione deve rispondere subito.

Esempio: un pulsante che aggiunge un prodotto a una lista e mostra un messaggio di conferma.

```
1.<!DOCTYPE html>
2.<html lang="it">
3.<head>
4.  <meta charset="UTF-8">
5.  <title>Esempio UX - Feedback</title>
6.  <link rel="stylesheet" href="style.css">
7.</head>
8.<body>
9.
10.  <div class="contenitore">
11.    <h1>Lista prodotti</h1>
12.
13.    <input type="text" id="prodotto" placeholder="Inserisci un prodotto">
14.
15.    <button onclick="aggiungiProdotto()">Aggiungi</button>
16.
17.    <p id="messaggio"></p>
18.
19.    <ul id="lista"></ul>
20.  </div>
21.
22.  <script src="script.js"></script>
23.</body>
24.</html>
```

```
1.body {
2.  font-family: Arial, sans-serif;
3.  background-color: #f4f4f4;
4.}
5.
6..contenitore {
7.  width: 400px;
8.  margin: 50px auto;
9.  background-color: white;
10.  padding: 25px;
11.  border-radius: 10px;
12.}
13.
14.input {
15.  width: 70%;
16.  padding: 10px;
17.}
18.
19.button {
20.  padding: 10px;
21.  cursor: pointer;
22.}
23.
24.#messaggio {
25.  font-weight: bold;
26.  color: green;
27.}
28.
29.li {
30.  margin-top: 8px;
31.}
```

```
1.function aggiungiProdotto() {
2.  // Recupero il valore scritto dall'utente
3.  let input = document.getElementById("prodotto");
4.  let valore = input.value;
5.
6.  // Recupero il messaggio e la lista
7.  let messaggio = document.getElementById("messaggio");
8.  let lista = document.getElementById("lista");
9.
10.  // Controllo se l'utente non ha scritto nulla
11.  if (valore === "") {
12.    messaggio.innerHTML = "Devi inserire un prodotto.";
13.    messaggio.style.color = "red";
14.  } else {
15.    // Creo un nuovo elemento della lista
16.    let elemento = document.createElement("li");
17.    elemento.innerHTML = valore;
18.
19.    // Aggiungo l'elemento alla lista
20.    lista.appendChild(elemento);
21.
22.    // Mostro un feedback positivo
23.    messaggio.innerHTML = "Prodotto aggiunto correttamente.";
24.    messaggio.style.color = "green";
25.
26.    // Svuoto il campo input
27.    input.value = "";
28.  }
29.}
```



Questo esempio migliora la UX perché l'utente riceve subito una risposta. Se il campo è vuoto, l'applicazione mostra un errore chiaro. Se invece l'azione è corretta, mostra un messaggio positivo.

Validazione semplice di un form

Un altro aspetto importante della UX è evitare che l'utente invii dati sbagliati senza spiegazioni.

Esempio: un form di registrazione che controlla nome ed email.

```
1.<!DOCTYPE html>
2.<html lang="it">
3.<head>
4.  <meta charset="UTF-8">
5.  <title>Esempio UX - Form</title>
6.  <link rel="stylesheet" href="style.css">
7.</head>
8.<body>
9.
10.  <div class="card">
11.    <h1>Registrazione</h1>
12.
13.    <label>Nome</label>
14.    <input type="text" id="nome" placeholder="Inserisci il nome">
15.
16.    <label>Email</label>
17.    <input type="text" id="email" placeholder="Inserisci l'email">
18.
19.    <button onclick="controllaForm()">Registrati</button>
20.
21.    <p id="risultato"></p>
22.  </div>
23.
24.  <script src="script.js"></script>
25.</body>
26.</html>
```

```
1.body {
2.  font-family: Arial, sans-serif;
3.  background-color: #e9eef3;
4.}
5.
6..card {
7.  width: 350px;
8.  margin: 60px auto;
9.  background-color: white;
10.  padding: 25px;
11.  border-radius: 12px;
12.}
13.
14.label {
15.  display: block;
16.  margin-top: 15px;
17.  font-weight: bold;
18.}
19.
20.input {
21.  width: 100%;
22.  padding: 10px;
23.  margin-top: 5px;
24.  box-sizing: border-box;
25.}
26.
27.button {
28.  width: 100%;
29.  margin-top: 20px;
30.  padding: 12px;
31.  cursor: pointer;
32.}
33.
34..errore {
35.  border: 2px solid red;
36.}
37.
38..corretto {
39.  border: 2px solid green;
40.}
41.
42.#risultato {
43.  font-weight: bold;
44.}
```

```
1.function controllaForm() {
2.  let nome = document.getElementById("nome");
3.  let email = document.getElementById("email");
4.  let risultato = document.getElementById("risultato");
5.
6.  // Tolgo eventuali classi precedenti
7.  nome.classList.remove("errore", "corretto");
8.  email.classList.remove("errore", "corretto");
9.
10.  // Controllo se il nome è vuoto
11.  if (nome.value === "") {
12.    nome.classList.add("errore");
13.    risultato.innerHTML = "Il nome è obbligatorio.";
14.    risultato.style.color = "red";
15.  }
16.  // Controllo se l'email non contiene la chiocciola
17.  else if (!email.value.includes("@")) {
18.    email.classList.add("errore");
19.    risultato.innerHTML = "Inserisci una email valida.";
20.    risultato.style.color = "red";
21.  }
22.  else {
23.    nome.classList.add("corretto");
24.    email.classList.add("corretto");
25.    risultato.innerHTML = "Registrazione completata.";
26.    risultato.style.color = "green";
27.  }
28.}
```



Il form non si limita a “non funzionare”, ma spiega all'utente cosa deve correggere. Il bordo rosso evidenzia il campo problematico, mentre il messaggio testuale chiarisce l'errore.

Stato attivo di una scelta

Nelle applicazioni è importante far capire all'utente quale opzione ha selezionato. Esempio: selezione di una categoria.

```
1.<!DOCTYPE html>
2.<html lang="it">
3.<head>
4.  <meta charset="UTF-8">
5.  <title>Esempio UX - Selezione</title>
6.  <link rel="stylesheet" href="style.css">
7.</head>
8.<body>
9.
10.  <div class="box">
11.    <h1>Scegli una categoria</h1>
12.
13.    <button class="categoria" onclick="selezionaCategoria(this)">Film</button>
14.    <button class="categoria" onclick="selezionaCategoria(this)">Libri</button>
15.    <button class="categoria" onclick="selezionaCategoria(this)">Videogiochi</button>
16.
17.    <p id="scelta"></p>
18.  </div>
19.
20.  <script src="script.js"></script>
21.</body>
22.</html>
```

```
1.body {
2.  font-family: Arial, sans-serif;
3.  background-color: #f7f7f7;
4.}
5.
6..box {
7.  width: 450px;
8.  margin: 60px auto;
9.  padding: 25px;
10.  background-color: white;
11.  border-radius: 10px;
12.}
13.
14..categoria {
15.  padding: 12px;
16.  margin: 5px;
17.  cursor: pointer;
18.  border: 1px solid #999;
19.  background-color: white;
20.}
21.
22..attiva {
23.  background-color: #222;
24.  color: white;
25.}
26.
27.#scelta {
28.  margin-top: 20px;
29.  font-weight: bold;
30.}
```

```
1.function selezionaCategoria(bottoneCliccato) {
2.  let bottoni = document.getElementsByClassName("categoria");
3.
4.  // Rimuovo la classe attiva da tutti i bottoni
5.  for (let i = 0; i < bottoni.length; i++) {
6.    bottoni[i].classList.remove("attiva");
7.  }
8.
9.  // Aggiungo la classe attiva solo al bottone cliccato
10.  bottoneCliccato.classList.add("attiva");
11.
12.  // Mostro la scelta dell'utente
13.  document.getElementById("scelta").innerHTML =
14.    "Hai scelto: " + bottoneCliccato.innerHTML;
15.}
```

L'utente capisce visivamente quale scelta è attualmente attiva. Senza questo feedback, potrebbe non sapere se il click è stato registrato.

Questo principio è usato spesso in menu, filtri, tab, dashboard e pannelli di configurazione



Bottone disabilitato fino a input valido

Un'applicazione può guidare l'utente impedendo azioni non valide. Esempio: il pulsante "Invia" si abilita solo quando il campo contiene testo.

```
1.<!DOCTYPE html>
2.<html lang="it">
3.<head>
4.  <meta charset="UTF-8">
5.  <title>Esempio UX - Bottone Disabilitato</title>
6.  <link rel="stylesheet" href="style.css">
7.</head>
8.<body>
9.
10.  <div class="contenitore">
11.    <h1>Invia commento</h1>
12.
13.    <textarea id="commento" placeholder="Scrivi un commento" oninput="controllaTesto()"></textarea>
14.
15.    <button id="invia" disabled onclick="inviaCommento()">Invia</button>
16.
17.    <p id="messaggio"></p>
18.  </div>
19.
20.  <script src="script.js"></script>
21.</body>
22.</html>
```

```
1.body {
2.  font-family: Arial, sans-serif;
3.  background-color: #eeeeee;
4.}
5.
6..contenitore {
7.  width: 400px;
8.  margin: 60px auto;
9.  background-color: white;
10.  padding: 25px;
11.  border-radius: 10px;
12.}
13.
14.textarea {
15.  width: 100%;
16.  height: 100px;
17.  padding: 10px;
18.  box-sizing: border-box;
19.}
20.
21.button {
22.  margin-top: 15px;
23.  padding: 12px;
24.  width: 100%;
25.  cursor: pointer;
26.}
27.
28.button:disabled {
29.  cursor: not-allowed;
30.  opacity: 0.5;
31.}
```

```
1.function controllaTesto() {
2.  let commento = document.getElementById("commento");
3.  let invia = document.getElementById("invia");
4.
5.  // Se il commento contiene testo, abilito il pulsante
6.  if (commento.value !== "") {
7.    invia.disabled = false;
8.  } else {
9.    invia.disabled = true;
10. }
11.}
12.
13.function inviaCommento() {
14.  document.getElementById("messaggio").innerHTML =
    "Commento inviato correttamente.";
15.}
```



Il bottone disabilitato comunica all'utente che manca ancora qualcosa. In questo modo si riducono errori inutili e messaggi di errore successivi.

È una UX più preventiva: invece di far sbagliare l'utente e poi correggerlo, lo guida prima.

Contatore caratteri

Un altro esempio molto comune è il contatore dei caratteri, utile nei commenti, nei post, nei messaggi o nei form.

```
1.<!DOCTYPE html>
2.<html lang="it">
3.<head>
4.  <meta charset="UTF-8">
5.  <title>Esempio UX - Contatore</title>
6.  <link rel="stylesheet" href="style.css">
7.</head>
8.<body>
9.
10.  <div class="area">
11.    <h1>Scrivi una descrizione</h1>
12.
13.    <textarea id="descrizione" maxlength="100" oninput="contaCaratteri()"></textarea>
14.
15.    <p id="contatore">0 / 100 caratteri</p>
16.  </div>
17.
18.  <script src="script.js"></script>
19.</body>
20.</html>
```

```
1.body {
2.  font-family: Arial, sans-serif;
3.  background-color: #f0f0f0;
4.}
5.
6.area {
7.  width: 400px;
8.  margin: 60px auto;
9.  background-color: white;
10. padding: 25px;
11. border-radius: 10px;
12.}
13.
14.textarea {
15.  width: 100%;
16.  height: 120px;
17.  padding: 10px;
18.  box-sizing: border-box;
19.}
20.
21.#contatore {
22.  text-align: right;
23.  color: #555;
24.}
```

```
1.function contaCaratteri() {
2.  let descrizione = document.getElementById("descrizione");
3.  let contatore = document.getElementById("contatore");
4.
5.  let numeroCaratteri = descrizione.value.length;
6.
7.  contatore.innerHTML = numeroCaratteri + " / 100 caratteri";
8.}
```

Il contatore aiuta l'utente a sapere quanto spazio ha ancora a disposizione.



Questo evita sorprese quando il testo è troppo lungo e rende il comportamento dell'applicazione più trasparente.

Conclusione

La UX applicativa serve a rendere l'interazione più chiara, sicura e prevedibile.

In HTML, CSS e JavaScript possiamo migliorarla con elementi semplici: messaggi di feedback, validazione dei form, stati visivi, bottoni disabilitati, contatori e conferme.

Anche senza framework, questi piccoli accorgimenti trasformano una pagina statica in una vera interfaccia applicativa più comoda da usare.



Conferma prima di eliminare

Le azioni distruttive, come cancellare un elemento, dovrebbero sempre essere chiare e possibilmente confermate.

```
1.<!DOCTYPE html>
2.<html lang="it">
3.<head>
4.  <meta charset="UTF-8">
5.  <title>Esempio UX - Eliminazione</title>
6.  <link rel="stylesheet" href="style.css">
7.</head>
8.<body>
9.
10.  <div class="contenitore">
11.    <h1>Archivio documenti</h1>
12.
13.    <div id="documento" class="documento">
14.      Documento importante.pdf
15.      <button onclick="eliminaDocumento()">Elimina</button>
16.    </div>
17.
18.    <p id="messaggio"></p>
19.  </div>
20.
21.  <script src="script.js"></script>
22.</body>
23.</html>
```

```
1.body {
2.  font-family: Arial, sans-serif;
3.  background-color: #f5f5f5;
4.}
5.
6..contenitore {
7.  width: 450px;
8.  margin: 60px auto;
9.  background-color: white;
10. padding: 25px;
11. border-radius: 10px;
12.}
13.
14..documento {
15.  padding: 15px;
16.  border: 1px solid #ccc;
17.}
18.
19..documento button {
20.  float: right;
21.  cursor: pointer;
22.}
```

```
1.function eliminaDocumento() {
2.  let conferma = confirm("Sei sicuro di voler eliminare il
  documento?");
3.
4.  if (conferma === true) {
5.    let documento =
      document.getElementById("documento");
6.    documento.remove();
7.
8.    document.getElementById("messaggio").innerHTML =
      "Documento eliminato correttamente.";
9.  }
10.}
11.}
```

L'eliminazione è un'azione rischiosa. Chiedere conferma evita cancellazioni accidentali.

Questo è particolarmente importante nelle applicazioni reali, come gestionali, dashboard, CRM, pannelli admin o applicazioni scolastiche.



Buon Davante a tutti

